

# RELATÓRIO DE AUDITORIA DE SEGURANÇA

## SmartParking - ParkingFAG

**Data:** 16/09/2026

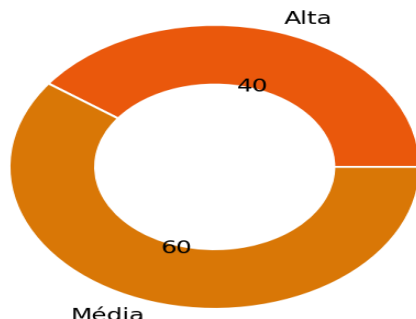
**Escopo:** código-fonte entregue no ZIP, frontend, backend, migração Supabase, Docker/Compose, documentação, testes e histórico Git incluído no pacote.

**Nota metodológica:** as cinco categorias solicitadas foram mapeadas para a stack detectada. Foram percorridos todos os handlers do backend, os sinks de HTML do frontend, configuração/deploy e os quatro commits presentes no histórico Git. Somente achados demonstráveis no código foram classificados como falha.

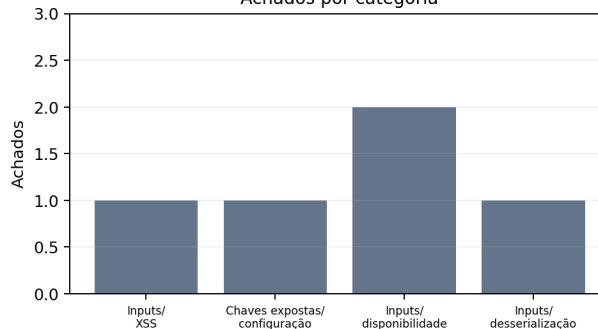
# 1. Resumo executivo

| Total | Crítica | Alta | Média | Baixa |
|-------|---------|------|-------|-------|
| 5     | 0       | 2    | 3     | 0     |

Achados por severidade



Achados por categoria



Os riscos centrais são a possibilidade de XSS armazenado no painel, a aceitação de uma chave de ingestão pública de exemplo quando não substituída, e riscos de disponibilidade/desserialização. Não foram verificadas falhas de IDOR ou de autorização por papel porque o projeto não possui usuários, tenants, papéis nem rotas por ID.

## 2. Stack e cobertura metodológica

### Stack

Python 3.12; backend HTTP/ASGI próprio (biblioteca padrão), sem Flask/FastAPI/Django; repositório REST manual para Supabase/PostgreSQL, sem ORM/query builder; autenticação de escrita por chave compartilhada X-Ingest-Key; frontend HTML/CSS/JavaScript vanilla; Dockerfile + compose.yaml; Vercel via `api/index.py`; migração SQL Supabase. Nenhum Helm/Terraform encontrado. O ZIP não contém `.github/workflows`, embora README mencione CI.

### 1. Banco sem tranca

Mecanismo detectado: RLS no Supabase + revogação de anon/authenticated, com `service_role` somente no backend. Não existe modelo de usuário/tenant/workspace no domínio; o painel é público por desenho. Nenhuma quebra de isolamento por tenant/owner foi encontrada. O GET latest usa `service_role`, mas expõe apenas a última ocupação pública prevista pelo produto.

### 2. Permissão no navegador

Não há papéis, `isAdmin/canEdit/role`, login ou UI administrativa no frontend. Portanto, não há gates de papel a cruzar. A única escrita é POST `/api/readings` e a checagem ocorre no servidor.

### 3. IDOR

Foram percorridos todos os handlers em `ParkingApplication.dispatch`: GET `/api/health`, GET `/api/readings/latest`, POST `/api/readings`, GET `/` e GET `/static/*`. Não existem rotas por ID nem objetos pertencentes a usuário/tenant; categoria não aplicável.

### 4. Chaves hardcoded

Working tree, configs, docs, compose/Docker e histórico Git disponível foram pesquisados. Não há segredo real hardcoded. Há, porém, um placeholder público de `PARKING_INGEST_KEY` que o runtime aceita como segredo real se copiado sem troca (Achado 2). Nenhum bundle/minificador: `static/app.js` foi inspecionado diretamente e não contém chave.

## 5. Inputs / XSS

Foi pesquisado innerHTML/equivalentes, eval/new Function, URLs dinâmicas e sanitização. Existe um sink innerHTML em static/app.js:10-14 e não há biblioteca de sanitização; combinado com spaces sem schema interno no backend, gera XSS armazenado (Achado 1). Não foram encontrados markdown, eval/new Function ou templates de e-mail.

## 3. Pontos fortes

- Supabase: RLS está habilitado em supabase/migrations/202609030001\_create\_parking\_readings.sql:15; anon e authenticated têm privilégios revogados nas linhas 17-19. O acesso privilegiado fica no backend via service\_role.
- POST /api/readings: app/web.py:63-67 exige X-Ingest-Key e retorna 503 se a chave não estiver configurada e 401 quando diverge; tests/system/test\_user\_flow.py:27-35 cobre o fluxo não autorizado/autorizado.
- Traversal de arquivos estáticos: app/web.py:77-84 resolve o caminho e rejeita candidatos fora de STATIC (linhas 79-81).
- Campos numéricos principais: app/domain.py:27-33 valida total/free e consistência da contagem; source é limitado a 100 caracteres na linha 39.
- Segredos reais: varredura do working tree e dos 4 commits disponíveis não encontrou chave Supabase, token, senha ou chave privada real commitida. .gitignore:3 ignora .env; README.md:74 orienta não versionar SUPABASE\_SECRET\_KEY.
- Frontend: free\_spaces, occupied\_spaces, total\_spaces, captured\_at e status usam textContent (static/app.js:5-8,24-28), evitando HTML nesses pontos.

## 4. Pontos fracos

Os pontos fracos verificados concentram-se em validação de entrada e hardening operacional: dados internos de spaces chegam ao DOM sem escaping; o placeholder da chave de ingestão pode ser aceito como segredo; não há limites explícitos de corpo/quantidade; e três utilitários carregam pickle sem autenticação do arquivo.

## 5. Achados detalhados

### 1. XSS armazenado no mapa de vagas via space.index

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                     |                                                |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|------------------------------------------------|
| <b>Alta</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | <b>Inputs / XSS</b> | static/app.js:10-14; app/domain.py:31-33,44-50 |
| <pre>// static/app.js:10-14 byId("spaces").innerHTML = spaces.map((space) =&gt; `   &lt;article class="space \${space.is_free ? "free" : "occupied"}"&gt;     &lt;strong&gt;Vaga \${space.index}&lt;/strong&gt;     &lt;span&gt;\${space.is_free ? "Livre" : "Ocupada"}&lt;/span&gt;   &lt;/article&gt;`).join("");  # app/domain.py:31-33,48 normalized_spaces = spaces or [] if normalized_spaces and len(normalized_spaces) != total_spaces:     raise ValueError(...) ... spaces=payload.get("spaces") or [],</pre> |                     |                                                |

**Por que é explorável:** O backend aceita os objetos internos de spaces sem validar o tipo/conteúdo de index. A leitura é persistida e depois retornada por /api/readings/latest. O navegador interpola space.index em innerHTML. Um cliente que possua a chave de ingestão pode gravar marcação HTML/JavaScript e executar código no navegador de quem abrir o painel. Condição: capacidade de publicar uma leitura (PARKING\_INGEST\_KEY válida).

**Correção:** Validar cada item de spaces no backend (index e pixel\_count inteiros; is\_free booleano) e construir o DOM com textContent/createElement, sem innerHTML para dados dinâmicos.

### 2. Chave pública de exemplo pode virar credencial real de ingestão

|                                                                                                                                                                                                                                                                                                                                   |                                       |                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|---------------------------------------------------|
| <b>Alta</b>                                                                                                                                                                                                                                                                                                                       | <b>Chaves expostas / configuração</b> | .env.example:5; README.md:35; app/web.py:41,64-67 |
| <pre># .env.example:5 PARKING_INGEST_KEY=troque-por-uma-chave-longa  # README.md:35 Copie `.env.example` para `.env` ...  # app/web.py:41,64-67 self.ingest_key = ... os.getenv("PARKING_INGEST_KEY", "") if not self.ingest_key: ... if headers.get("x-ingest-key") != self.ingest_key:     return json_response(401, ...)</pre> |                                       |                                                   |

**Por que é explorável:** A documentação orienta copiar o arquivo de exemplo, e o servidor só rejeita chave vazia. Se o operador não substituir o placeholder conhecido, 'troque-por-uma-chave-longa' passa a ser uma credencial válida e pública. Qualquer pessoa que conheça o repositório pode publicar leituras. Condição: deploy realizado com o placeholder sem sobrescrita.

**Correção:** Falhar no startup quando PARKING\_INGEST\_KEY estiver ausente, curta ou igual a placeholders conhecidos; documentar geração criptograficamente aleatória.

### 3. Corpo HTTP é lido integralmente antes da autenticação, sem limite

| Média                                                                                                                                                                                                                                                                                                         | Inputs / disponibilidade | server.py:10-14; app/web.py:94-105 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|------------------------------------|
| <pre># server.py:10-14 length = int(self.headers.get("Content-Length", "0")) body = self.rfile.read(length) if length else b"" response = application.dispatch(...)  # app/web.py:94-105 chunks = [] ... chunks.append(message.get("body", b"")) ... response = parking.dispatch(..., b"".join(chunks))</pre> |                          |                                    |

**Por que é explorável:** Tanto o servidor local quanto o adaptador ASGI acumulam o corpo completo em memória antes de ParkingApplication verificar X-Ingest-Key. Um cliente não autenticado pode enviar corpos muito grandes e consumir memória/conexões. Exploração depende de o endpoint estar alcançável e de limites do proxy/plataforma não bloquearem antes.

**Correção:** Aplicar limite de Content-Length/body no primeiro ponto de entrada, rejeitando com 413 antes de acumular o corpo; manter limites também no proxy/plataforma.

### 4. Leitura aceita quantidade de vagas sem limite superior

| Média                                                                                                                                                                                                                                                                                   | Inputs / disponibilidade | app/domain.py:27-33,44-50 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|---------------------------|
| <pre># app/domain.py:27-33 if total_spaces &lt; 0:     raise ValueError(...) if not 0 &lt;= free_spaces &lt;= total_spaces:     raise ValueError(...) normalized_spaces = spaces or [] if normalized_spaces and len(normalized_spaces) != total_spaces:     raise ValueError(...)</pre> |                          |                           |

**Por que é explorável:** Há validação de consistência, mas nenhum teto para total\_spaces nem para a lista spaces. Um publicador autenticado pode enviar uma leitura extremamente grande, elevando CPU, memória, tráfego e armazenamento no Supabase. Condição: PARKING\_INGEST\_KEY válida (ou exploração combinada com o achado 2).

**Correção:** Definir MAX\_SPACES compatível com o estacionamento e rejeitar total\_spaces/listas acima do limite; limitar também tamanho e campos de cada item.

### 5. Desserialização insegura de CarParkPos com pickle.load

| Média                                                                                                                                                                                                                                                                                                                   | Inputs / desserialização | main.py:97-98; Parkingspace picker.py:6-10; maintrackbars.py:8-9 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|------------------------------------------------------------------|
| <pre># main.py:97-98 with (ROOT / "CarParkPos").open("rb") as positions_file:     positions = pickle.load(positions_file)  # Parkingspace picker.py:6-8 with open('CarParkPos', 'rb') as f:     posList = pickle.load(f)  # maintrackbars.py:8-9 with open('CarParkPos', 'rb') as f:     posList = pickle.load(f)</pre> |                          |                                                                  |

**Por que é explorável:** pickle executa instruções de desserialização e não é seguro para conteúdo não confiável. Se CarParkPos puder ser substituído (arquivo baixado, artefato adulterado, máquina compartilhada ou comprometimento da cadeia de entrega), abrir qualquer um desses scripts pode executar código arbitrário com os privilégios do usuário. Condição: atacante precisa conseguir adulterar CarParkPos antes da execução.

**Correção:** Substituir pickle por JSON/CSV com validação estrita de uma lista de pares inteiros; se legado for inevitável, não aceitar arquivos externos e verificar integridade/autenticidade antes de carregar.

## 6. Recomendações priorizadas

**P1** - Remover o sink innerHTML para dados dinâmicos e validar o schema de cada vaga no backend.

**P1** - Rejeitar PARKING\_INGEST\_KEY de exemplo/fraca no startup e rotacionar qualquer ambiente que tenha usado o placeholder.

**P2** - Impor limites de body e MAX\_SPACES, com testes para 413/422 e cargas extremas.

**P2** - Migrar CarParkPos de pickle para JSON/CSV validado.

**P3** - Adicionar testes de segurança regressivos: payload XSS, chave placeholder, corpo acima do limite e schema inválido.

## 7. ISSUES PARA O GITHUB

Os blocos abaixo estão prontos para copiar e colar. Achados de limite de body e limite de vagas foram agrupados para evitar spam.

```

--- ISSUE 1 ---
# [Segurança] Eliminar XSS armazenado no mapa de vagas

**Labels sugeridas:** security, alta

## Descrição
O backend aceita os objetos internos de spaces sem validar o tipo/conteúdo de index. A leitura é
persistida e depois retornada por /api/readings/latest. O navegador interpola space.index em
innerHTML. Um cliente que possua a chave de ingestão pode gravar marcação HTML/JavaScript e executar
código no navegador de quem abrir o painel. Condição: capacidade de publicar uma leitura
(PARKING_INGEST_KEY válida).

## Evidência
static/app.js:10-14; app/domain.py:31-33,44-50
// static/app.js:10-14
byId("spaces").innerHTML = spaces.map((space) => `
  <article class="space ${space.is_free ? "free" : "occupied"}">
    <strong>Vaga ${space.index}</strong>
    <span>${space.is_free ? "Livre" : "Ocupada"}</span>
  </article>`).join("");

# app/domain.py:31-33,48
normalized_spaces = spaces or []
if normalized_spaces and len(normalized_spaces) != total_spaces:
    raise ValueError(...)
...
spaces=payload.get("spaces") or [],

## Impacto
Execução de JavaScript no contexto do painel, permitindo manipular a interface e agir com as
permissões/origem do site.

## Sugestão de correção
Validar cada item de spaces no backend (index e pixel_count inteiros; is_free booleano) e construir
o DOM com textContent/createElement, sem innerHTML para dados dinâmicos.

## Critérios de aceite
- [ ] Nenhum dado de `spaces` é inserido via `innerHTML`.
- [ ] `index`/`pixel_count` aceitam apenas inteiros e `is_free` apenas booleano.
- [ ] Payload com `<img onerror=...>` não executa script.
- [ ] Teste automatizado cobre o caso.
--- FIM ISSUE 1 ---

--- ISSUE 2 ---
# [Segurança] Rejeitar PARKING_INGEST_KEY padrão ou fraca

**Labels sugeridas:** security, alta

## Descrição
A documentação orienta copiar o arquivo de exemplo, e o servidor só rejeita chave vazia. Se o
operador não substituir o placeholder conhecido, 'troque-por-uma-chave-longa' passa a ser uma
credencial válida e pública. Qualquer pessoa que conheça o repositório pode publicar leituras.
Condição: deploy realizado com o placeholder sem sobrescrita.

## Evidência
.env.example:5; README.md:35; app/web.py:41,64-67
# .env.example:5
PARKING_INGEST_KEY=troque-por-uma-chave-longa

# README.md:35
Copie `.env.example` para `.env` ...

# app/web.py:41,64-67
self.ingest_key = ... os.getenv("PARKING_INGEST_KEY", "")
if not self.ingest_key: ...
if headers.get("x-ingest-key") != self.ingest_key:
    return json_response(401, ...)

## Impacto
Publicação não autorizada de leituras e possível encadeamento com o XSS armazenado.

## Sugestão de correção
Falhar no startup quando PARKING_INGEST_KEY estiver ausente, curta ou igual a placeholders
conhecidos; documentar geração criptograficamente aleatória.

## Critérios de aceite
- [ ] Startup/deploy rejeita `troque-por-uma-chave-longa`.
- [ ] Chaves abaixo do comprimento/política definida são rejeitadas.
- [ ] Documentação ensina gerar segredo aleatório.
- [ ] Teste automatizado cobre placeholder e chave válida.
--- FIM ISSUE 2 ---

--- ISSUE 3 ---
# [Segurança] Limitar tamanho de requisições e leituras de estacionamento

```

```

**Labels sugeridas:** security, média

## Descrição
Tanto o servidor local quanto o adaptador ASGI acumulam o corpo completo em memória antes de
ParkingApplication verificar X-Ingest-Key. Um cliente não autenticado pode enviar corpos muito
grandes e consumir memória/conexões. Exploração depende de o endpoint estar alcançável e de limites
do proxy/plataforma não bloquearem antes. Além disso, Há validação de consistência, mas nenhum teto
para total_spaces nem para a lista spaces. Um publicador autenticado pode enviar uma leitura
extremamente grande, elevando CPU, memória, tráfego e armazenamento no Supabase. Condição:
PARKING_INGEST_KEY válida (ou exploração combinada com o achado 2).

## Evidência
server.py:10-14; app/web.py:94-105
# server.py:10-14
length = int(self.headers.get("Content-Length", "0"))
body = self.rfile.read(length) if length else b""
response = application.dispatch(...)

# app/web.py:94-105
chunks = []
...
chunks.append(message.get("body", b""))
...
response = parking.dispatch(..., b"".join(chunks))

app/domain.py:27-33,44-50
# app/domain.py:27-33
if total_spaces < 0:
    raise ValueError(...)
if not 0 <= free_spaces <= total_spaces:
    raise ValueError(...)
normalized_spaces = spaces or []
if normalized_spaces and len(normalized_spaces) != total_spaces:
    raise ValueError(...)

## Impacto
Consumo excessivo de memória/CPU/tráfego/armazenamento e degradação ou indisponibilidade do serviço.

## Sugestão de correção
Aplicar limite de body antes da autenticação/acúmulo e definir MAX_SPACES/schema com limites de
tamanho.

## Critérios de aceite
- [ ] Corpo acima do limite retorna 413 sem ser acumulado integralmente.
- [ ] `total_spaces` acima de MAX_SPACES retorna 422.
- [ ] `spaces` acima do limite é rejeitado.
- [ ] Testes cobrem servidor HTTP e camada de aplicação.
--- FIM ISSUE 3 ---

--- ISSUE 4 ---
# [Segurança] Substituir pickle por formato seguro em CarParkPos

**Labels sugeridas:** security, média

## Descrição
pickle executa instruções de desserialização e não é seguro para conteúdo não confiável. Se
CarParkPos puder ser substituído (arquivo baixado, artefato adulterado, máquina compartilhada ou
comprometimento da cadeia de entrega), abrir qualquer um desses scripts pode executar código
arbitrário com os privilégios do usuário. Condição: atacante precisa conseguir adulterar CarParkPos
antes da execução.

## Evidência
main.py:97-98; Parkingspace picker.py:6-10; maintrackbars.py:8-9
# main.py:97-98
with (ROOT / "CarParkPos").open("rb") as positions_file:
    positions = pickle.load(positions_file)

# Parkingspace picker.py:6-8
with open('CarParkPos', 'rb') as f:
    posList = pickle.load(f)

# maintrackbars.py:8-9
with open('CarParkPos', 'rb') as f:
    posList = pickle.load(f)

## Impacto
Execução arbitrária de código local se o arquivo CarParkPos for adulterado.

## Sugestão de correção
Substituir pickle por JSON/CSV com validação estrita de uma lista de pares inteiros; se legado for
inevitável, não aceitar arquivos externos e verificar integridade/autenticidade antes de carregar.

## Critérios de aceite
- [ ] Nenhum dos três scripts usa `pickle.load` para CarParkPos.
- [ ] Formato substituto aceita somente coordenadas inteiras válidas.
- [ ] Arquivo malformado falha de forma segura.
- [ ] Testes cobrem leitura válida e inválida.
--- FIM ISSUE 4 ---

```

**Fim do relatório.**